Maw3ed— No-Show Reducer PRD

Oct 28, 2026 · @Ali Obeid

Overview

Maw3ed (working name) is a multi-business booking page with automatic reminders, a waitlist and per-client no-show history, built to prove one number: a lower no-show rate during the pilot than in the owner's own book for the two weeks before.

Who it's for. Salons, barbers, dental and beauty clinics, and physiotherapists. Most owners will only open it on a phone.

The pain. Every no-show is a paid hour lost. Today reminders depend on a receptionist remembering to send them, so they go out late, unevenly, or not at all.

What version one does.

- Publishes a booking page per business, built from its services, staff and working hours.
- Reminds every client 24 hours and 2 hours before, by email automatically, plus a one-tap WhatsApp send for the receptionist.
- Puts confirm and cancel links in every reminder, so a client who can't come frees the slot instead of vanishing.
- Offers any freed slot to the waitlist.
- Keeps a no-show history per client.

Engineering heart. Slots generated from staff hours, no double booking when two requests arrive at once, and scheduled reminder jobs that never send twice.

Constraints. Team of two, three weeks, Standard level. Review checks after 8 November 2026. No health records: clinic pilots store appointments only.

Goals, non-goals and success metrics

The pilot succeeds if each pilot business's no-show rate falls below its own two-week baseline, with zero double bookings and zero duplicate reminders along the way.

Goals

1. Lower the no-show rate for every pilot business against its own book.

2. Take reminders off the receptionist's memory: every eligible appointment gets both reminders without anyone remembering to send them.
3. Turn cancellations into refilled slots through the waitlist.
4. Pass every item of the 8 November definition of done.

Metric	Definition	Target	Source
No-show rate	No-shows ÷ (attended + no-shows), per business per week. Cancellations via link are excluded.	Below the two-week baseline for each pilot business; proposed stretch: 30% relative drop	Appointment outcomes marked by staff; baseline from owner's book
Reminder coverage	Appointments booked 24 h+ ahead that received the 24 h email ÷ all such appointments	100%	Reminder job log
Duplicate reminders	Same appointment, same reminder kind, same channel sent more than once	0	Reminder job log + provider IDs
Double bookings	Two active appointments overlapping for one staff member	0	Database constraint + nightly check query
Confirm rate	Reminders answered with Confirm ÷ reminders sent	Tracked, no target	Link clicks
Waitlist refill rate	Freed slots rebooked from the waitlist ÷ freed slots offered	Tracked, no target	Waitlist offers

Non-goals for version one

- Health records, treatment notes or anything clinical. Clinic pilots store appointments only.
- Automated WhatsApp sending through the WhatsApp Business API. Version one uses a one-tap prefilled link the receptionist sends.

- SMS reminders.
- Rescheduling as its own action. A client cancels and rebooks.
- Point of sale, inventory, payroll, or multi-location chains.
- Native mobile apps. The web app must feel right on a phone instead.
- Real money. Deposits and checkout run only in test environments.

Personas and roles

Two signed-in roles, Owner and Staff, meet the review's two-role requirement; clients never create an account and act only through signed links.

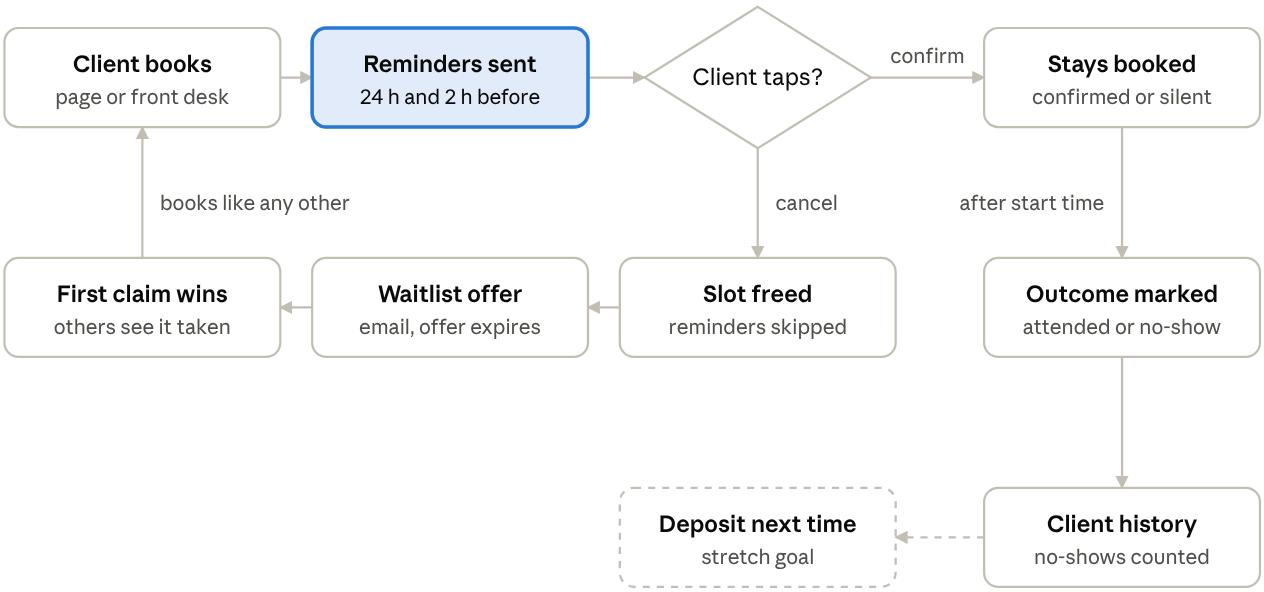
Role	Who	Can do	Cannot do
Owner	Salon owner, clinic manager, head barber	Sign up the business; manage services, staff, working hours, time off and settings; see every appointment, client and report; invite and remove staff; enter the pilot baseline; manage the plan	—
Staff	Stylists, practitioners and the receptionist	See the business calendar; book phone and walk-in clients; send due WhatsApp reminders; mark attended, no-show or late cancel; view client no-show history	Change services, hours, settings, billing or staff accounts; see another business
Client (no account)	The person booking	Book from the public page; confirm or cancel through a reminder link; join the waitlist; claim an offered slot	See anything beyond their own appointment

A staff member has a `bookable` flag: practitioners are bookable, a receptionist is not. Owners can also be bookable, since many barbers and therapists own the shop and work the chair.

Core user flows

The core flow is the one to build first: a client books, gets reminders with confirm and cancel links, and a freed slot goes to the waitlist.

Every booking gets reminders; every cancellation feeds the waitlist



core flow · booking, reminders, waitlist, outcome

A waitlist claim re-enters at the top as an ordinary booking, so it gets reminders and the same double-booking protection.

Owner setup, done once on a phone:

1. Sign up with business name, booking-page slug and timezone.
2. Add services with duration, buffer and price.
3. Add staff, mark who is bookable, and tick the services each one performs.
4. Set weekly hours per staff member and any time off.
5. Share the booking-page link on Instagram, WhatsApp status and Google Maps.
6. Invite the receptionist as Staff.

Receptionist's day: open the dashboard, send the WhatsApp reminders due now with one tap each, book phone and walk-in clients, and mark each finished appointment's outcome.

Functional requirements

Twenty-four of the 28 requirements below are Must or Should and make up version one; the four Could rows are scored extras. Priorities are dropdowns, so re-rank them as the build progresses.

ID	Area	Requirement and acceptance	Priority
FR-01	Workspace	Any business signs up with name, slug, timezone and owner account; a workspace is created and the owner lands on a setup checklist.	Must
FR-02	Auth	Email and password sign-in with Owner and Staff roles; the owner invites staff by an expiring email link.	Must
FR-03	Isolation	Every read and write is scoped to the session's business; another business's IDs return 404, never data.	Must
FR-04	Setup	Owner manages services: name, duration (min), buffer after (min), price, active.	Must
FR-05	Setup	Owner manages staff and which services each one performs.	Must
FR-06	Setup	Weekly working hours per staff member, several intervals per day, plus dated time off.	Must
FR-07	Setup	Business settings: timezone (default Asia/Beirut), slot step, booking window in days, minimum notice, cancellation cutoff.	Must
FR-08	Booking page	Public page at /b/{slug}: service → staff or "anyone" → day → slot → name, phone, email → booked, in four screens or fewer on a phone.	Must

ID	Area	Requirement and acceptance	Priority
FR-09	Booking page	Slots come from staff hours minus appointments, buffers and time off; none in the past or inside minimum notice.	Must
FR-10	Booking page	When requests for the same slot arrive together, exactly one succeeds; the rest get 409 and fresh slots.	Must
FR-11	Front desk	Staff book phone and walk-in clients from the dashboard through the same slot engine.	Must
FR-12	Booking page	Booking confirmation email with the confirm and cancel links and an add-to-calendar file.	Should
FR-13	Reminders	Email reminders 24 h and 2 h before start, each with confirm and cancel links.	Must
FR-14	Reminders	Front-desk “Reminders due” list with a one-tap WhatsApp button that opens a prefilled message with both links and records who sent it.	Must
FR-15	Reminders	Cancelled appointments get no further reminders; one booked under 24 h ahead skips the 24 h reminder; one booked under 2 h ahead gets none.	Must
FR-16	Reminders	Each reminder (appointment × kind × channel) is sent at most once, even with retries, overlapping runs or crashes.	Must
FR-17	Links	Confirm and cancel links are signed, tied to one appointment, work without sign-in and stop working at start time; cancelling after the cutoff records a late cancel.	Must
FR-18	Waitlist	When nothing fits, a client joins the waitlist for a service, a date range and optionally a staff member.	Must
FR-19	Waitlist	A freed slot is offered by email to matching waitlist entries; the first to claim gets it through the same no-double-booking path; the offer expires.	Must
FR-20	Waitlist	An unclaimed offer cascades to the next batch until the slot is too close to fill.	Should

ID	Area	Requirement and acceptance	Priority
FR-21	History	Staff mark each past appointment Attended, No-show or Late cancel.	Must
FR-22	History	A client record per business, keyed by phone, shows visits, no-shows and late cancels; repeat no-showers carry a badge on every booking.	Must
FR-23	History	Appointments past their end with no outcome appear in a “Mark outcome” queue.	Should
FR-24	Pilot	Owner enters the two-week baseline (appointments and no-shows) from their book; a pilot view shows weekly no-show rate against it, plus confirm and refill rates.	Must
FR-25	Pilot	CSV export of appointments and outcomes.	Could
FR-26	Stretch	Deposit required for clients with past no-shows, through Whish Money's test environment.	Could
FR-27	Scored extra	Pricing page with a test-mode checkout.	Could
FR-28	Scored extra	Arabic interface with right-to-left layout.	Could

Non-functional requirements

Tenant isolation, correctness under concurrency and phone-first usability are the three that the review and the pilot will actually test.

ID	Area	Requirement	How it is verified
NFR-01	Tenant isolation	Every repository method takes `businessId` from the session, never from the request body. Public routes resolve the business from the slug.	Integration test: business A's owner requests B's appointments, clients, services and staff by ID and gets 404 each time

ID	Area	Requirement	How it is verified
NFR-02	Validation	A Zod schema on every route's params, query and body; unknown fields rejected; errors return 400 with field messages.	Route test per endpoint with a bad payload
NFR-03	Auth and security	Passwords hashed with bcrypt or argon2; httpOnly, Secure, SameSite=Lax session cookie; role checks in the service layer; link tokens signed with HMAC and scoped to one appointment and action.	Unit tests on the guard and token helpers
NFR-04	Privacy	Store only name, phone, email, appointment time, service and staff. No free-text notes on clinic appointments. A client can be deleted on request.	Schema review; no notes column exists
NFR-05	Phone first	Designed at 360 px wide first; tap targets of 44 px or more; booking in four screens or fewer; the owner dashboard usable one-handed.	Manual check on a real Android phone each week; Lighthouse mobile accessibility 90+
NFR-06	Time	Store UTC; compute and display in the business's timezone. Lebanon changes clocks twice a year, so slot generation must be correct across a clock change.	Unit tests on the days around a clock change
NFR-07	Concurrency	No two active appointments overlap for one staff member, enforced by the database, not only by application code.	Parallel-request integration test (section below)

ID	Area	Requirement	How it is verified
NFR-08	Reminder reliability	A reminder job runs at least every 5 minutes; a missed run catches up on the next; no reminder is ever sent twice.	Job tests with two overlapping runs and a simulated crash
NFR-09	Performance	Slots for one service across 14 days for a 10-person business return in under 300 ms at p95.	Seeded load test in CI (non-blocking)
NFR-10	Email limits	Volume fits the free email tier: Resend's free plan is reported at 3,000 emails a month and 100 a day (source), so about 33 appointments a day across all pilots at 3 emails each.	Daily count in the job log, alert at 80
NFR-11	Observability	Structured logs with business ID and request ID; every reminder attempt logged with its provider message ID; error tracking on a free tier.	Visible in the demo
NFR-12	Internationalisation ready	All interface strings in message files from day one, so Arabic and right-to-left can be added without touching components.	Lint rule against hard-coded strings in components

Engineering heart

All three hard problems are solved in the database or in pure functions, so they can be proven with tests rather than argued in the demo.

1. Slot generation

A pure function `generateSlots(input) → Slot[]` with no database access, so it is fully unit-tested.

Inputs: service duration and buffer; each eligible staff member's weekly intervals and dated time off; their busy intervals (active appointments, each as start to end plus buffer); slot step; minimum notice; booking window; business timezone; `now`.

Steps for each day and each eligible staff member:

1. Build that day's working intervals in local time and convert them to UTC instants with the business timezone, so a clock-change day comes out right.
2. Subtract time off and busy intervals.
3. Walk each free interval at the slot step; emit a start when start + duration + buffer fits inside the interval and start ⩾ now + minimum notice.
4. For "anyone", merge all staff: a start time shows if at least one person is free. On booking, assign the free person with the fewest bookings that day.

Unit tests cover: split shifts with a lunch break; a service longer than the remaining interval; buffers; time off; minimum notice; the booking window edge; the days around a clock change; "anyone" merging.

2. No double booking

The database is the referee. A Postgres exclusion constraint rejects any overlapping active appointment for the same staff member, so no race in application code can get past it.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "Appointment"
  ADD CONSTRAINT appointment_no_overlap
  EXCLUDE USING gist (
    "staffId" WITH =,
    tstzrange("startAt", "blockedUntil", '[)') WITH &&
  ) WHERE (status IN ('BOOKED', 'CONFIRMED'));
```

- Prisma cannot express this constraint, so it lives in a hand-written SQL migration, documented in the README's key decisions.
- `blockedUntil` = end + buffer, so buffers are protected too. Cancelling sets the status to CANCELLED, which drops the row out of the constraint and frees the slot.

- The service re-checks the slot first to give a friendly error, then inserts. A violation (Postgres code 23P01) maps to 409 `SLOT_TAKEN` with fresh slots in the body.
- `POST /bookings` takes an `Idempotency-Key` header, so a double-tapped Book button returns the first booking instead of failing.
- Waitlist claims go through exactly the same insert, so two people claiming one offer are handled identically.
- **Proof:** an integration test fires 10 parallel requests for one slot and asserts exactly one 201 and nine 409s.

3. Reminder jobs that never send twice

Every reminder is a row before it is a message. The row's unique key makes "send twice" impossible to record, the claim step makes it impossible to pick up twice, and the provider's idempotency key covers a crash between sending and saving.

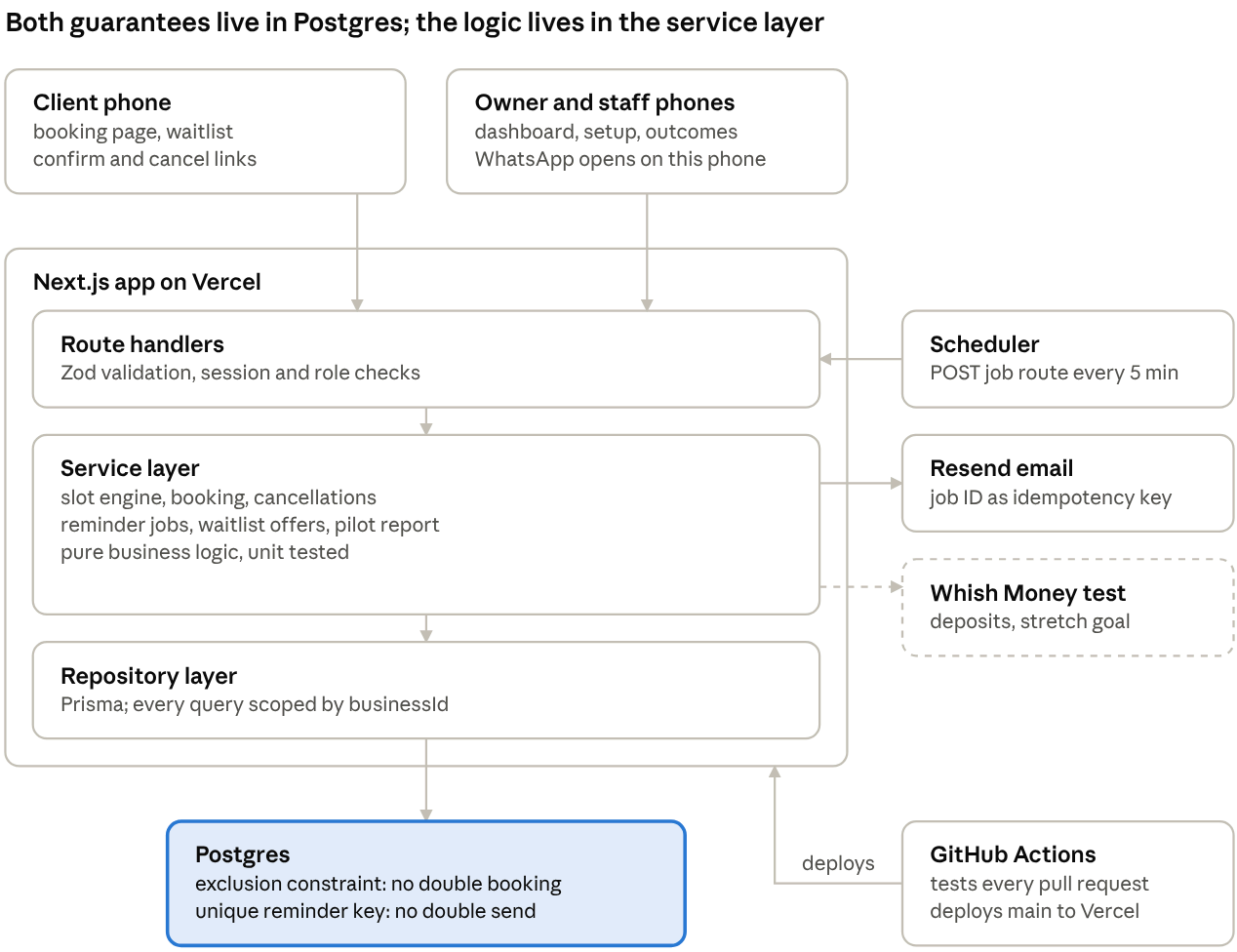
1. **Create.** Booking an appointment inserts `ReminderJob` rows: 24 h email, 2 h email, 24 h WhatsApp, 2 h WhatsApp, each with its `dueAt`. A unique key on (appointmentId, kind, channel) stops duplicates. Kinds already in the past at booking are not created.
2. **Trigger.** A protected route, `POST /api/jobs/reminders`, runs every 5 minutes. Vercel's free Hobby plan only allows cron jobs once a day (Vercel docs), so an external scheduler calls it with a shared secret. Because the run works on "whatever is due", a late or doubled trigger is harmless.
3. **Claim.** One statement moves due rows from PENDING to SENDING with `FOR UPDATE SKIP LOCKED`, so two overlapping runs never claim the same row.
4. **Re-check.** If the appointment is cancelled, or the send window has passed (24 h reminder after the 2 h mark; 2 h reminder within 15 minutes of start), mark SKIPPED.
5. **Send.** Email goes out with the job's ID as the provider idempotency key. Resend returns the original result for a repeated key within 24 hours (Resend docs), so retrying a job stuck in SENDING after a crash cannot deliver a second email.
6. **Record.** Mark SENT with the provider message ID. Failures go back to PENDING with an attempt count and backoff, and to FAILED after 3 attempts.
7. **WhatsApp.** Due WhatsApp rows show in the front-desk list instead of being sent by the server. Tapping opens a prefilled `wa.me` message and marks the row SENT with who sent it; a second receptionist sees "sent by Rana, 09:14" and the button disabled.

```
UPDATE "ReminderJob" SET status = 'SENDING', "lockedAt" = now()
WHERE id IN (
  SELECT id FROM "ReminderJob"
  WHERE status = 'PENDING' AND channel = 'EMAIL' AND "dueAt" <= now()
  ORDER BY "dueAt" LIMIT 50
  FOR UPDATE SKIP LOCKED
)
RETURNING *;
```

Proof: tests run two job runs at once against the same due rows and assert one send each; cancel an appointment and assert both reminders are SKIPPED; simulate a crash after sending and assert the retry reuses the same idempotency key.

Architecture and stack

One Next.js app on free tiers: route handlers validate and authorise, the service layer holds the logic, and repositories are the only code that touches Prisma.



architecture · three layers, Postgres, four outside services

Neither WhatsApp nor the scheduler needs a paid integration: WhatsApp is a prefilled link opened on the receptionist's phone, and any free scheduler can call the job route.

Concern	Choice	Why
Framework	Next.js App Router, route handlers, TypeScript strict	Accelerator stack; one deployable
Data	Prisma on Postgres (Neon or Supabase free tier)	Accelerator stack; Postgres gives the exclusion constraint and `SKIP LOCKED`. Confirm `btree_gist` can be enabled on the chosen host
Validation and contract	Zod on every route; OpenAPI generated from the same schemas	One source of truth for checks and docs

Concern	Choice	Why
Auth	Auth.js credentials provider or a small session table; role on Membership	Two roles, cookie sessions, no third-party sign-in needed
Email	Resend, behind an `EmailProvider` interface	Free tier and idempotency keys
Jobs	`POST /api/jobs/reminders` called every 5 minutes by an external scheduler	Vercel Hobby cron runs only once a day
UI	Tailwind and shadcn/ui, phone-first; next-intl for strings	Fast to build; ready for Arabic
Tests and CI	Vitest, Playwright, GitHub Actions with a Postgres service	Matches the definition of done

```
src/
  app/api/…            route handlers: parse with Zod, call one service, map
errors
  server/domain/       pure functions: generateSlots, reminder windows,
tokens
  server/services/     booking, reminders, waitlist, outcomes, pilot
  server/repositories/ Prisma access, businessId required on every method
  server/providers/    EmailProvider, PaymentProvider and their fakes
prisma/
  schema.prisma
  migrations/…_appointment_no_overlap/migration.sql   hand-written constraint
```

Data model

Fifteen tables, every tenant table carrying `businessId` as the first column of its indexes; the repository's ERD is generated from this Prisma schema.

Entity	Key fields	Belongs to	Constraints and notes
Business	slug, name, timezone, slotStepMin, bookingWindowDays, minNoticeMin, cancelCutoffMin	—	slug unique; the tenant root
User	email, passwordHash, name	—	email unique; one person can belong to several businesses
Membership	role (OWNER, STAFF)	User, Business	unique (userId, businessId)
Invite	email, role, tokenHash, expiresAt	Business	single use
StaffMember	displayName, bookable, active	Business, optional User	a receptionist has a User but is not bookable
Service	name, durationMin, bufferMin, price, active	Business	soft-deactivate, never delete
StaffService	—	StaffMember, Service	join table
WorkingHours	weekday, startLocal, endLocal	StaffMember	local times such as 09:00; several rows per day allowed
TimeOff	startAt, endAt	StaffMember	UTC instants
Client	name, phone, email, attended, noShows, lateCancels	Business	unique (businessId, phone); counters updated in the same transaction as the outcome

Entity	Key fields	Belongs to	Constraints and notes
Appointment	startAt, endAt, blockedUntil, status, source, idempotencyKey	Business, Client, StaffMember, Service	exclusion constraint on staff and time while BOOKED or CONFIRMED; status also ATTENDED, NO_SHOW, CANCELLED, LATE_CANCEL
ReminderJob	kind (H24, H2), channel (EMAIL, WHATSAPP), dueAt, status, attempts, lockedAt, sentAt, sentBy, providerMessageId	Appointment	unique (appointmentId, kind, channel); index (status, dueAt)
WaitlistEntry, WaitlistOffer	service, optional staff, date range, status; offer startAt, expiresAt, claimedAppointmentId	Business, Client	one open offer per entry per freed slot
PilotBaseline	fromDate, toDate, appointments, noShows	Business	entered by the owner from their own book

The stretch goal adds a `Payment` table (appointment, provider, external ID, amount, status) and a `requiresDeposit` flag derived from the client's no-show count.

API surface

About 30 route handlers in three groups: public (by slug or signed token), signed-in (scoped to the session's business), and the job trigger. The OpenAPI 3.1 contract is generated from the same Zod schemas the routes validate with, so the two cannot drift.

Method	Path	Access	Purpose
POST	/api/auth/signup	Public	Create business, owner user and membership

Method	Path	Access	Purpose
POST	/api/auth/login, /api/auth/logout	Public / signed in	Session cookie
POST	/api/invites, /api/invites/{token}/accept	Owner / public	Invite staff
GET	/api/public/{slug}	Public	Business name, services, bookable staff
GET	/api/public/{slug}/slots? serviceId&staffId&from&to	Public	Available slots
POST	/api/public/{slug}/bookings	Public, Idempotency-Key	Book; 201 or 409 SLOT_TAKEN
POST	/api/public/{slug}/waitlist	Public	Join the waitlist
GET, POST	/api/links/{token}	Signed token	Show the appointment; confirm or cancel
POST	/api/offers/{token}/claim	Signed token	Claim a waitlist offer
CRUD	/api/services, /api/staff, /api/staff/{id}/hours, /api/staff/{id}/time-off	Owner	Setup
GET, PATCH	/api/business	Owner	Settings
GET, POST	/api/appointments	Owner, Staff	Calendar list; front-desk booking
PATCH	/api/appointments/{id}/outcome	Owner, Staff	Attended, no-show, late cancel
POST	/api/appointments/{id}/cancel	Owner, Staff	Cancel on the client's behalf; triggers the waitlist
GET	/api/reminders/whatsapp-due	Owner, Staff	Front-desk list with prefilled links

Method	Path	Access	Purpose
POST	/api/reminders/{id}/mark-sent	Owner, Staff	Record a WhatsApp send
GET	/api/clients, /api/clients/{id}	Owner, Staff	Client list and no-show history
GET, PUT	/api/pilot/baseline, GET /api/pilot/report	Owner	Pilot measurement
POST	/api/jobs/reminders, /api/jobs/waitlist-expiry	Shared secret	Scheduled runs

Errors share one shape: `{ "error": { "code": "SLOT_TAKEN", "message": "…", "fields": {…} } }`.

Stretch goal and scored extras

Build these only after the core flow passes its tests at the end of week two; they share one payment adapter, so the deposit and the pricing checkout cost one integration, not two.

Deposit for past no-showers (stretch)

1. When a client whose record shows one or more no-shows books, the appointment is created as PENDING_PAYMENT and holds the slot for 15 minutes. PENDING_PAYMENT is added to the exclusion constraint's active statuses so the hold is real.
2. The client is sent to a Whish Money checkout in the test environment for the deposit the owner set.
3. On return, the server verifies the payment status with Whish directly, never trusting the redirect alone, then moves the appointment to BOOKED and creates its reminder jobs.
4. A job cancels unpaid holds after 15 minutes and offers the slot to the waitlist.

We found no public Whish Money merchant API documentation, so test credentials and endpoint details should come from the accelerator or Whish merchant onboarding. Code talks to a `PaymentProvider` interface with a `WhishTestProvider` and a `FakeProvider`; CI and local development use the fake.

Pricing page with test checkout (scored)

A public /pricing page with two plans, for example Free (one bookable staff member) and Pro (unlimited staff, deposits). Upgrade runs a checkout through the same `PaymentProvider` in test mode and sets the business's plan on success.

Arabic interface (scored)

Strings already live in message files (NFR-12), so this is translation plus layout: `dir="rtl"` on the root, logical CSS properties (start and end, not left and right), and Arabic email and WhatsApp templates chosen per business. The booking page and the WhatsApp message matter most, because that is what clients see.

Testing and CI/CD

Every pull request runs lint, type-check, unit and integration tests against a real Postgres in CI; merging to main deploys to production only when all of them pass.

Layer	Tool	What it covers
Unit	Vitest	`generateSlots`, reminder due-time and window rules, token signing and expiry, no-show counters, waitlist matching, role guards
Integration	Vitest + Postgres service container + route handlers called directly	Booking happy path; 10 parallel bookings → one 201; tenant isolation (B cannot read A); confirm and cancel links; cancel → waitlist offer → claim; two overlapping reminder runs → one send each; validation 400s on every route
End to end (smoke)	Playwright, phone viewport	Book from the public page; owner marks a no-show
Contract	OpenAPI generated from Zod, checked in CI	The committed contract matches the routes

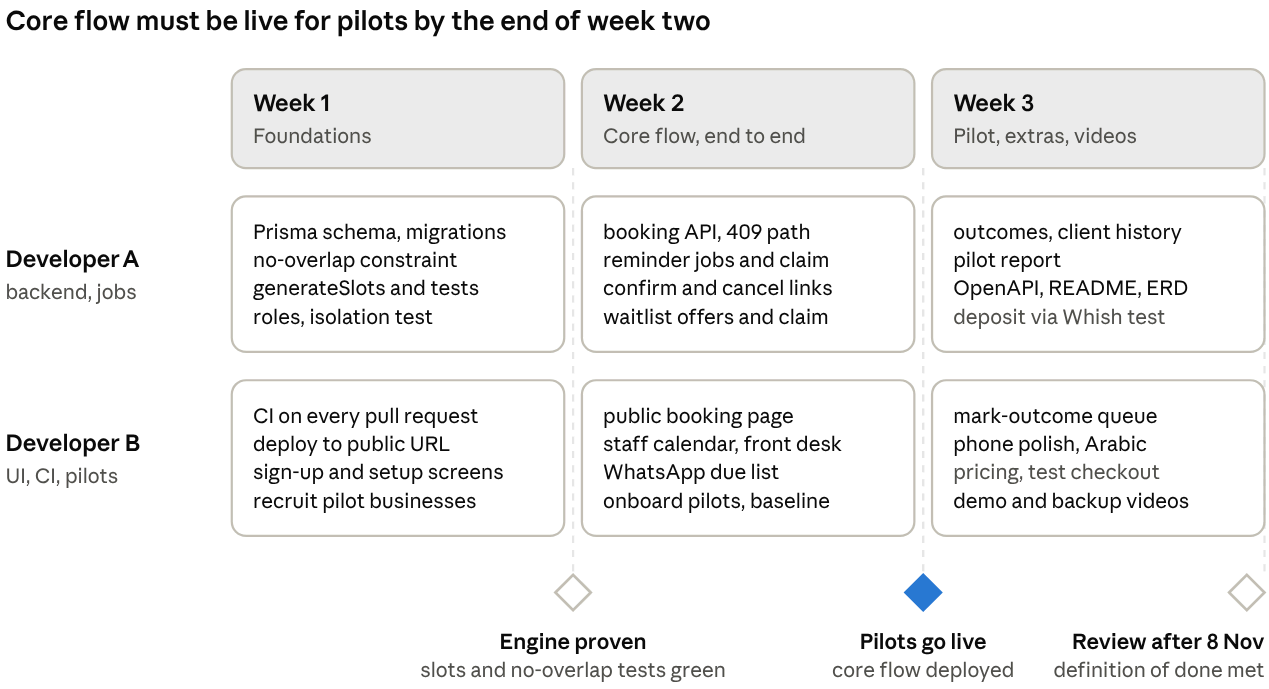
Pipeline (GitHub Actions)

1. On every pull request: install → Prisma migrate against the Postgres service → lint → type-check → unit → integration → build. Branch protection blocks merge on any failure.
2. On merge to main: the same checks, then `prisma migrate deploy` against production and a production deploy, so the public URL is always the tested main branch.
3. Each pull request gets a preview deployment for review on a phone.

Email and payment providers are swapped for fakes in tests through the same interfaces used in production; the fakes record calls, which is how “sent exactly once” is asserted.

Three-week plan

Week one proves the engine, week two ships the core flow to real businesses, and week three runs the pilot while the extras are added.



three-week plan · two developers, three gates

Grey items are scored extras and the stretch goal; drop them first if the core slips. Recruiting starts in week one because the pilot needs every possible day before the review.

Pilot plan and measurement

Run the pilot with three to five businesses of different types, and compare each one only with its own book for the two weeks before it started.

Recruit. At least one salon or barber and one clinic, so the demo shows both a walk-in-heavy and an appointment-heavy book. Each owner agrees to put every appointment into the system during the pilot, including phone bookings, or the comparison breaks.

Onboard in one 20-minute visit. Create the business, services, staff and hours with the owner on their phone; enter the next week's existing appointments from their book; show the receptionist the WhatsApp list.

Baseline. The owner counts, from their own book for the 14 days before go-live, total appointments and no-shows, and enters them on the pilot page (FR-24). A no-show means the client neither came nor cancelled before the cutoff; late cancels are counted separately in both periods so the definitions match.

During the pilot. Staff mark every appointment's outcome the same day; the "Mark outcome" queue nags until they do. Check the job log daily for skipped or failed reminders.

Report per business:

Field	Example of what is shown
Baseline	appointments, no-shows, rate
Pilot	appointments, no-shows, rate, days covered
Change	absolute points and relative %
Supporting signals	confirm rate, cancellations via link, waitlist refills

With a few dozen appointments per business, a single no-show moves the rate by several points. The report therefore always shows the raw counts next to the rate, and the demo claims a direction, not statistical significance.

Risks, open questions and definition of done

The biggest risk is not code but time: the pilot needs real days of data before the 8 November review, so businesses must be live by the start of week three at the latest.

Risk	Effect	Mitigation
Pilot starts too late	Too few days to compare with the baseline	Recruit during week one; go live with the core flow at the end of week two; cut extras before cutting pilot days
Owners keep using their paper book	Pilot numbers miss appointments	Front-desk booking in under 30 seconds; enter existing bookings during onboarding
Email lands in spam or clients don't read email	Reminders don't reach clients	Verified sending domain with SPF and DKIM; WhatsApp one-tap as the main channel for many clients
Free email cap of 100 a day	Reminders stop mid-pilot	Count sends daily; drop the booking confirmation email before any reminder if near the cap
Receptionist ignores the WhatsApp list	WhatsApp reminders not sent	Badge count on the dashboard; the list is the first screen staff see
Exclusion constraint missed by a Prisma migration	Double bookings possible	Hand-written SQL migration plus the parallel-booking test in CI
External scheduler late or down	Reminders late	Due-window logic catches up on the next run; a second scheduler as backup is harmless because runs are idempotent
Whish test credentials arrive late	Stretch goal slips	Fake provider first; the stretch goal is optional

Open questions

- [ ] Does the three-week build window end on 8 November? If so, which date did it start, and how many pilot days fit before the review?

- [ ] Which external scheduler: GitHub Actions schedule, cron-job.org, or a queue service? (Must call the job route every 5 minutes on a free tier.)
- [ ] Who provides the sending domain for email?
- [ ] Who provides Whish Money test credentials and documentation?
- [ ] Deposit amount and no-show threshold: set per business by the owner, or one default?
- [ ] Phone format: store E.164 (+961 …) and normalise local formats on entry?

Definition of done (review after 8 November)

- [ ] Live at a public URL, deployed by a CI pipeline that runs the tests on every pull request
- [ ] Next.js route handlers in TypeScript and Prisma; repository layer separate from service layer; validation on every route
- [ ] Any business can sign up to its own workspace; a test proves one business cannot read another's data
- [ ] Sign-in with Owner and Staff roles
- [ ] Comfortable on a phone
- [ ] Repository with README (setup, architecture, key decisions), FR and NFR one-pager, ERD and OpenAPI contract
- [ ] Unit tests on business logic; integration tests on the main endpoints
- [ ] 3-minute demo video and 60-second backup video
- [ ] Scored extras attempted: pricing page with test checkout, Arabic interface, deposit stretch goal